TALLER PRÁCTICO DE SISTEMAS DISTRIBUIDOS Y RPC

Integración de JavaScript Vainilla con ZeroC Ice sobre WebSockets Bidireccionales

Compilación de Contratos Slice a JS, Invocación Remota RPC y Callbacks en Tiempo Real (`BiDir`) sin Polling

FACULTAD DE INGENIERÍA, DISEÑO Y CIENCIAS APLICADAS — DEPARTAMENTO TIC

Curso:	Computación en Internet I (09776)	**Semestre:**	2026-2
Profesor:	Alejandro Muñoz M.	**Duración Estimada:**	2 Horas
Estudiante 1:	_____	**Código 1:**	_____
Estudiante 2:	_____	**Código 2:**	_____

1 Contexto del Problema y Estado Inicial del Proyecto

El repositorio base de este taller contiene una plataforma distribuida de **Ajedrez Multijugador** sobre **ZeroC Ice 3.7**. El proyecto ya cuenta con:

1. **Módulo** `common/`**:** Define el contrato Slice (`ChessService.ice`), el motor autoritativo de reglas de ajedrez (`ChessRules.java`) y el gestor de clasificación por Tiers (`TierManager.java`).
2. **Módulo** `server/`**:** Servidor ZeroC Ice en Java que expone simultáneamente un endpoint TCP (`default -p 10000`) y un endpoint WebSocket (`ws -h 0.0.0.0 -p 10002`).
3. **Módulo** `client/`**:** Cliente de escritorio en **JavaFX** completamente funcional que se conecta vía TCP al puerto `10000`.
4. **Módulo** `client-js/`**:** Interfaz web componetizada en **JavaScript Vainilla** que construye el DOM dinámicamente sobre un único contenedor (`src/app.js` y `src/components/`).

Objetivo del Taller: Actualmente, la capa de servicios RPC del cliente web en `client-js/src/services/` se encuentra incompleta. Su misión es conectar el frontend en JavaScript Vainilla con el servidor ZeroC Ice mediante WebSockets, compilar y enlazar el contrato Slice `ChessService.js`, e implementar la actualización del tablero en tiempo real mediante **Conexiones Bidireccionales (*BiDir*)** sin utilizar *polling*.

2 Contrato Slice de Referencia (`common/src/main/slice/ChessService.ice`)

Revisar las interfaces y estructuras definidas en el contrato Slice que deberán ser invocadas e implementadas desde JavaScript:

```
module ChessApp {
    enum PieceColor { COLORWHITE, COLORBLACK, COLORNONE };
    enum PieceType  { PIECEPAWN, PIECEKNIGHT, PIECEBISHOP, PIECEROOK, PIECEQUEEN, PIECEKING, PIECEEMPTY };
    enum PlayerRole { ROLEWHITE, ROLEBLACK, ROLESPECTATOR };
    enum GameStatus { WAITINGFORPLAYERS, INPROGRESS, CHECKWHITE, CHECKBLACK,
                      CHECKMATEWHITEWINS, CHECKMATEBLACKWINS, STALEMATE, RESIGNEDWHITEWINS, RESIGNEDBLACKWINS,
                          ABORTED };
    enum TierLevel  { TIERBRONZE, TIERSILVER, TIERGOLD, TIERPLATINUM, TIERDIAMOND };

    struct Position { int row; int col; };
    struct ChessPiece { PieceType type; PieceColor color; };

    interface ChessObserver {
        void onGameStateChanged(GameState newState);
        void onGameFinished(GameState finalState, string winnerNickname, int pointsDelta);
    };

    interface ChessService {
        PlayerProfile getProfile(string nickname);
        PlayerLeaderboard getLeaderboard();
        string createRoom(string creatorNickname);
        JoinRoomResult joinRoom(string roomCode, string nickname) throws RoomNotFoundException, ChessException;
```

```
        void registerObserver(string roomCode, string nickname, ChessObserver* observer);
        void unregisterObserver(string roomCode, string nickname);
        GameState getGameState(string roomCode) throws RoomNotFoundException;
        bool makeMove(string roomCode, string nickname, Position fromPos, Position toPos)
            throws InvalidMoveException, NotPlayerTurnException, RoomNotFoundException;
        void resign(string roomCode, string nickname) throws RoomNotFoundException, UnauthorizedActionException;
        void leaveRoom(string roomCode, string nickname) throws RoomNotFoundException;
        RoomList getActiveRooms();
    };
};
```

3 Pliego de Requerimientos y Procedimiento de Implementación

El desarrollo del taller no consiste en copiar y pegar código prefabricado, sino en **satisfacer un pliego de requerimientos funcionales y contratos de arquitectura distribuida** entre el servidor Java y el cliente Web en JavaScript Vainilla.

Paso 1: Generación y Enlace del Contrato Slice (`ChessService.js`)

Para que el cliente web conozca las estructuras de datos (`Position`, `GameState`, `MoveHistoryItem`), el proxy del servicio (`ChessServicePrx`) y la interfaz abstracta del observador (`ChessObserver`), es indispensable compilar el contrato Slice:

1. Instalar las dependencias de Node.js en `client-js/`: npm install.
2. Compilar el archivo Slice a JavaScript ejecutando:

```
npx slice2js ../common/src/main/slice/ChessService.ice --output-dir src/services
# O mediante la tarea automatizada de Gradle desde la raiz: ./gradlew generateSliceJs
```

3. Verificar que `client-js/src/services/ChessService.js` haya sido generado correctamente. Al cargarse en la aplicación junto con `lib/Ice.min.js`, este archivo debe exponer el espacio de nombres global `window.ChessApp`.

Paso 2: Especificación BiDir en el Servidor Java (`ChessServiceImpl.java`)

Fundamento del Requerimiento: Por restricciones de seguridad del navegador web, una aplicación en JavaScript no puede abrir puertos de escucha TCP locales. En consecuencia, el proxy del observador enviado por el cliente web carece de endpoints (observer.ice_getEndpoints().length == 0).

- **Requerimiento:** En el método `registerObserver(roomCode, nickname, observer, current)`, verificar si el proxy recibido carece de endpoints asignados y si la conexión de transporte entrante (`current.con`) está activa.
- **Comportamiento Esperado:** Si carece de endpoints, fijar el proxy a la conexión entrante actual mediante observer.ice_fixed(current.con) antes de registrarlo en la sala (`GameRoom`). Si ya posee endpoints (como en los clientes de escritorio JavaFX), registrarlo directamente.

Paso 3: Especificación del Observador en Tiempo Real (`chessRealtimeObserver.js`)

Implementar la clase `ChessRealtimeObserver` para recibir eventos *push* del servidor sin utilizar *polling*:

- **Servant del Observador:** Definir una clase servant que herede de `this.ChessApp.ChessObserver` e implemente los métodos del contrato Slice:
  - `onGameStateChanged(newState, _current)`: Disparar el callback `onStateChanged` suministrando el nuevo `GameState`.
  - `onGameFinished(finalState, winnerNickname, pointsDelta, _current)`: Disparar el callback `onGameFinished`.
- **Ciclo de Vida del ObjectAdapter Bidireccional (`subscribe`):**
  1. Limpiar cualquier adaptador previo invocando `cleanupAdapter()`.
  2. Crear un `ObjectAdapter` local sin endpoints (communicator.createObjectAdapter("")), registrar el servant bajo una identidad única generada con UUID, y activar el adaptador.
  3. Obtener la conexión WebSocket activa del proxy del servicio (servicePrx.ice_getConnection()) y asociarle el adaptador local (connection.setAdapter(this.adapter)).
  4. Obtener el proxy no tipado del observador desde el adaptador, realizar un `uncheckedCast` a `ChessObserverPrx` y registrarlo en el servidor invocando servicePrx.registerObserver(roomCode, nickname, observerPrx).
- **Desuscripción y Limpieza (`unsubscribe` y `cleanupAdapter`):** Notificar al servidor mediante `unregisterObserver`, remover la identidad del adaptador, destruir el adaptador local con await this.adapter.destroy() y reestablecer las referencias a `null`.

Paso 4: Especificación del Servicio Cliente de Ice (`chessClientService.js`)

Implementar la fachada de comunicación RPC de la aplicación web:

- **Inicialización y Conexión (`connect`):** Configurar las propiedades de inicialización del `Communicator`: tamaño de mensaje máximo en `"2048"`, `Ice.ACM.Client.Timeout` en `"0"` y `Ice.ACM.Client.Heartbeat` en `"3"`. Resolver el proxy remoto mediante la cadena ChessService:ws –h <host> –p <port> y validar la conectividad con `checkedCast`.
- **Consultas Concurrentes de Lobby (`getLobbyData`):** Consultar concurrentemente las salas activas (`getActiveRooms`) y el ranking de jugadores (`getLeaderboard`) en el servidor aprovechando `Promise.all`.
- **Acciones de Sala y Partida:** Implementar la creación de sala (`createRoom`), unión a sala (`joinRoom`), rendición (`resign`) y abandono (`leaveRoom`).
- **Ejecución de Jugadas (`makeMove`):** Recibir las coordenadas numéricas `(fromRow, fromCol, toRow, toCol)`, construir dos instancias `Position` del contrato Slice e invocar remotamente el método `makeMove` del servicio Ice.

Paso 5: Especificación de Componentes DOM en JavaScript Vainilla (`client-js/src/components/`)

Para cerrar el ciclo distribuido desde el transporte hasta la interfaz gráfica de usuario, se debe implementar la manipulación reactiva del DOM en los siguientes componentes:

5.1 Renderizado del Tablero de Ajedrez (`ChessBoard.js` → `render`)

- **Entradas:** `gameState` (objeto `GameState` de Slice), `selectedSquare` (coordenada { row, col } o `null`), `legalMoves` (arreglo de destinos válidos).
- **RF-CB-01 (Validación y Grilla 8x8):** Si `gameState` o `gameState.board` no existen, abortar sin error. Recorrer las 64 casillas ($r \in [0,7], c \in [0,7]$), limpiar los hijos previos y restablecer la clase base (tile light o tile dark según $(r+c)\,\%2 == 0$).
- **RF-CB-02 (Resaltado de Jugada y Selección):** Si `gameState.moves` contiene jugadas, aplicar la clase CSS last–move a las casillas de origen y destino del último movimiento. Si (r, c) coincide con `selectedSquare`, aplicar la clase selected.
- **RF-CB-03 (Dibujo de Piezas):** Si `board[r][c]` contiene una pieza activa (piece.type._name !== ”PIECEEMPTY”), crear un con clase piece–white o piece–black según su color, asignar como texto el glifo Unicode devuelto por getPieceGlyph(piece) e insertarlo en la casilla.
- **RF-CB-04 (Sugerencias de Movimiento):** Si (r, c) figura en `legalMoves`, insertar un elemento indicador: con clase capture–ring si la casilla contiene pieza rival, o con clase move–dot si la casilla está vacía.

5.2 Historial de Jugadas por Turnos (`MoveHistory.js`)

- **RF-MH-01 (Estructura y Estado Vacío):** Construir el contenedor principal (moves–card), el encabezado fijo (player–card–header con texto *”Historial de Jugadas”*) y la lista desplazable (moves–list). Si el arreglo `moves` está vacío o es nulo, mostrar un con el texto exacto *”Sin jugadas aún”*.
- **RF-MH-02 (Agrupación de Turnos):** Iterar el arreglo `moves` (`MoveHistoryItem[]`) de dos en dos. Para cada turno, renderizar una fila con el número de turno en negrita (1. , 2. , etc.), seguido por la notación de blancas y, si existe, la de negras separadas por un espacio.
- **RF-MH-03 (Auto-Scroll):** Ajustar listContainer.scrollTop = listContainer.scrollHeight tras cada actualización para mantener visible la última jugada.

5.3 Tarjetas de Información de Jugadores (`PlayerCard.js` → `update`)

- **RF-PC-01 (Nombre y Puntuación):** Mostrar el `nickname` recibido (o el nombre por defecto si es nulo) y el puntaje formateado como ”<puntos> pts” (usar 1200 por defecto).
- **RF-PC-02 (Insignia de Tier y Capturas):** Actualizar la insignia de nivel invocando tierBadge.update(calculateTierFromPoints(puntos)). Si existen piezas capturadas (`capturedPieces`), renderizar sus glifos ordenados y separados por espacio; de lo contrario, vaciar el contenedor.

Paso 6: Verificación de Interoperabilidad en Vivo (JavaFX + Web Vanilla JS + Espectador)

1. Iniciar el servidor ZeroC Ice desde la raíz: `./gradlew runServer`.
2. Iniciar un cliente de escritorio JavaFX: `./gradlew runClient`, conectarse y crear una sala con el usuario `Alice`.

3. Iniciar el servidor web en `client-js/`: `npm start`, abrir `http://localhost:3000`, conectarse a `127.0.0.1:10002` y unirse a la sala de `Alice` con el usuario `Bob`.
4. Abrir una segunda pestaña del navegador o un segundo cliente JavaFX, unirse al mismo código con el usuario `Charlie` y verificar que recibe el rol **MODO ESPECTADOR**.
5. Realizar movimientos entre `Alice` (TCP) y `Bob` (WebSocket) y comprobar que los tres tableros, tarjetas e historiales se actualizan instantáneamente por eventos *push* sin usar *polling*.

4 Cuestionario de Análisis Conceptual y Arquitectura Distribuida

1. **Conexiones Bidireccionales (*BiDir*) vs. Callbacks Tradicionales:** ¿Por qué un cliente ejecutándose en el navegador web requiere crear un `ObjectAdapter` sin endpoints (createObjectAdapter("")) y asociarlo a `servicePrx.ice_getConnection()`, y qué papel cumple `observer.ice_fixed(current.con)` en el servidor Java?
2. **Transporte TCP vs. WebSockets en ZeroC Ice:** ¿Cómo logra el mismo servant `ChessServiceImpl` en Java atender simultáneamente a clientes JavaFX por TCP (`10000`) y clientes JavaScript por WebSockets (`10002`) sin duplicar la lógica del negocio?
3. **Empaquetado del Contrato con `slice2js`:** ¿Cómo registra el archivo generado `ChessService.js` el espacio de nombres `ChessApp` en el navegador y por qué es indispensable importar primero `Ice.min.js`?
4. **Validación Autoritativa:** Aunque el frontend en JavaScript calcula sugerencias visuales de casillas candidatas (`chessRulesHelper.js`), ¿por qué el servidor vuelve a validar cada movimiento en `GameRoom.makeMove`?

5 Rúbrica de Evaluación Modularizada por Punto (Escala 0.0 a 5.0)

Criterio Evaluativo	Pts	Excelente (4.5 – 5.0)	Aceptable (3.0 – 4.4)	Insuficiente (0.0 – 2.9)
1. Compilación y Enlace de `ChessService.js`	**0.5**	Genera `ChessService.js` con `slice2js`, lo ubica en `src/services/` y lo importa adecuadamente junto a `Ice.min.js`.	Genera el contrato pero requiere ajustes manuales en las rutas de importación.	No genera el archivo `ChessService.js` o falla al cargar el módulo `ChessApp`.
2. Soporte `BiDir` **en Servidor Java**	**0.7**	Actualiza `ChessServiceImpl.registerObserver` usando `observer.ice_fixed(current.con)` para proxies sin endpoints y explica su fundamento.	Aplica `ice_fixed` sin validar endpoints vacíos o sin comprender el flujo bidireccional.	El servidor no fija la conexión WebSocket y falla al notificar a los clientes web.
3. Servicio RPC `ChessClientService`	**1.0**	Implementa `connect` (`checkedCast` por `ws`), `getLobbyData`, `createRoom`, `joinRoom`, `makeMove`, `resign` y `leaveRoom`.	Conecta al servidor e invoca métodos del lobby pero presenta fallos al enviar coordenadas `Position` en `makeMove`.	No logra inicializar el `Communicator` ni establecer conexión con el endpoint WebSocket `10002`.
4. Observador en Tiempo Real (`ChessRealtimeObserver`**)**	**1.0**	Extiende `ChessApp.ChessObserver`, configura el `ObjectAdapter` sin endpoints sobre `ice_getConnection()` y actualiza el estado en vivo sin *polling*.	Registra el observador pero omite la limpieza en `unsubscribe` o depende parcialmente de *polling*.	No implementa el servant `ChessObserver` o utiliza `setInterval` (*polling*) en lugar de eventos Ice.
5. Componentes DOM en JS Vainilla	**1.0**	Implementa satisfactoriamente los requerimientos de `ChessBoard.render`, `MoveHistory` y `PlayerCard`, sincronizando datos remotos de Slice con el DOM.	Implementa el tablero pero omite el historial o la tarjeta de jugador, o presenta inconsistencias visuales menores.	No implementa la lógica de renderizado del DOM o rompe la presentación de las piezas y jugadas.
6. Interoperabilidad y Sustentación	**0.8**	Demuestra partida completa en tiempo real entre JavaFX, navegador web y espectador, respondiendo con rigor el cuestionario conceptual.	La partida funciona entre clientes pero existen errores menores al finalizar el juego o respuestas incompletas.	No hay interoperabilidad funcional entre el cliente JavaFX y el cliente JavaScript.
CALIFICACIÓN TOTAL	**5.0**	**Escala Institucional Universidad Icesi: Aprobatoria $\geq$ 3.0 • Duración: 2 Horas**		

INSTRUCCIONES DE ENTREGA

REPOSITORIO GIT

1. Confirmar y enviar todos los cambios al repositorio del equipo (git add . && git commit –m ”feat: implementacion cliente js y componentes dom con zeroc ice” && git pu
2. Verificar que el archivo generado `client-js/src/services/ChessService.js`, los servicios de red (`chessClientService.js`, `chessRealtimeObserver.js`), los componentes DOM (`ChessBoard.js`, `MoveHistory.js`, `PlayerCard.js`) y la modificación en `ChessServiceImpl.java` estén incluidos en el commit.
3. Demostrar en vivo una partida conectando un jugador desde JavaFX, un jugador desde el navegador web y un tercer participante en modo espectador.